SQL `LAG()` and `LEAD()` Window Functions

`LAG()` and `LEAD()` allow you to access values from previous and next rows, respectively, without performing a self-join.

They are commonly used for:

- Comparing current and previous month's sales.
- Finding salary changes.
- Detecting gaps in dates.
- Comparing consecutive transactions.
- Calculating growth or percentage change.

1. Sample table

Suppose we have monthly sales:

Month	Sals
January	1000
February	1500
March	1200
April	1800
May	2000

2. LAG() — Access previous row

```
SELECT
    month,
    sales,
    LAG(sales) OVER (
        ORDER BY month_number
    ) AS previous_sales
FROM monthly_sales;
```

Result:

Month	Sales	Previous_Sales
January	1000	NULL
February	1500	1000
March	1200	1500
April	1800	1200
May	2000	1800

How does it work?

- January has no previous row, so the result is NULL.
- February gets January's sales.
- March gets February's sales.
- And so on.

3. LEAD() — Access next row

```
SELECT
    month,
```

```
    sales,
    LEAD(sales) OVER (
        ORDER BY month_number
    ) AS next_sales
FROM monthly_sales;
```

Result:

Month	Sals	Next_Sales
January	1000	1500
February	1500	1200
March	1200	1800
April	1800	2000
May	2000	NULL

The last row has no next row, so it returns NULL.

4. Compare current sales with previous sales

This is a very common interview problem.

```
SELECT
    month,
    sales,
    LAG(sales) OVER (
        ORDER BY month_number
    ) AS previous_sales,
    sales - LAG(sales) OVER (
        ORDER BY month_number
    ) AS sales_difference
```

FROM monthly_sales;

Result:

Month	Sales	Previous Sales	Difference
January	1000	NULL	NULL
February	1500	1000	500
March	1200	1500	-300
April	1800	1200	600
May	2000	1800	200

You can also calculate percentage growth:

```
SELECT
    month,
    sales,
    ROUND(
        (sales - previous_sales) * 100.0 / previous_sales,
        2
    ) AS growth_percentage
FROM (
    SELECT
        month,
        sales,
        LAG(sales) OVER (
            ORDER BY month_number
        ) AS previous_sales
    FROM monthly_sales
);
```

5. Important: `LAG()` with offset and default value

The syntax is:

```
LAG(expression, offset, default_value)
OVER (
   PARTITION BY column
   ORDER BY column
)
```

Example:

```
SELECT
   month,
   sales,
   LAG(sales, 2, 0) OVER (
      ORDER BY month_number
   ) AS sales_two_months_ago
FROM monthly_sales;
```

Here:

- `sales` is the column to retrieve.
- `2` means look two rows back.
- `0` is the default if no such row exists.

Result:

Month	Sales	Sales_Two_Months_Ago
January	1000	0
February	1500	0
March	1200	1000

April	1800	1500
May	2000	1200

6. `LAG()` with `PARTITION BY`

Suppose you have sales from multiple departments:

Department	Month	Sales
IT	Jan	1000
IT	Feb	1500
IT	Mar	1200
HR	Jan	500
HR	Feb	700
HR	Mar	900

Query:

```
SELECT
    department,
    month,
    sales,
    LAG(sales) OVER (
```

```
        PARTITION BY department
        ORDER BY month_number
    ) AS previous_sales
FROM monthly_sales;
```

Result:

Department	Month	Sales	Previous Sales
IT	Jan	1000	NULL
IT	Feb	1500	1000
IT	Mar	1200	1500
HR	Jan	500	NULL
HR	Feb	700	500
HR	Mar	900	700

Notice that HR's January does not get IT's March value.

`PARTITION BY` resets the calculation for each department.

7. Common interview problems

Problem	Function

Compare current salary with previous salary	`LAG()`
Find next transaction date	`LEAD()`
Calculate month-over-month growth	`LAG()`
Find consecutive login dates	`LAG()`
Identify gaps between transactions	`LAG()`
Compare current and next event timestamps	`LEAD()`
Find employees whose salary increased	`LAG()`

8. Quick comparison

Feature	LAG	LEAD
Direction	Previous rows	Following rows
Default offset	1	1
Default value	NULL	NULL

Supports PARTITION BY	Yes	Yes
Supports ORDER BY	Yes	Yes

Remember this simple rule:

LAG()

Look backward

LEAD()

Look forward

Practice question

Suppose you have this transaction table:

Transaction ID	Customer ID	Transaction Date	Amount
1	C1	Jan 1	100
2	C1	Jan 5	200
3	C1	Jan 10	150
4	C2	Jan 2	300
5	C2	Jan 8	500

Question: Write an SQL query to display each customer's current transaction amount and their previous transaction amount.

Hint: Use `LAG()` with `PARTITION BY customer_id` and `ORDER BY transaction_date`.

This is a good interview exercise because it combines two important window-function concepts